

Contents

Open5GS 5G core + UERANSIM — end-to-end guide (with problems solved)	1
Part A — Deploy the Open5GS 5G SA core (summary)	1
Part B — Attach UERANSIM (gNB + UE)	2
Networking idea	2
gNB config — ~/UERANSIM/config/open5gs-gnb.yaml	2
UE config — ~/UERANSIM/config/open5gs-ue.yaml (key fields)	2
Start it	3
Part C — Problems solved (and why)	3
Problem 1 — Authentication failure (MAC failure): OP vs OPc mismatch	3
Problem 2 — UE traffic short-circuits when UE + gNB are on one host	3
Problem 3 — DNS inside the UE namespace (for the speed test)	4
Problem 4 (non-issue, worth knowing) — SQN out of range	4
Why Open5GS was easy where SD-Core was hard	4
Part D — Run the end-to-end tests	4
Part E — Cheat sheet / fresh-deploy checklist	5
Gotchas	5

Open5GS 5G core + UERANSIM — end-to-end guide (with problems solved)

A from-scratch guide to standing up the Open5GS 5G SA core and getting a real end-to-end data session (gNB + UE → internet) through it with UERANSIM. It includes every problem hit during bring-up and exactly how it was fixed, so a fresh deployment goes smoothly.

Companion artifacts in this repo: - Core deploy doc: [docs/open5gs-deployment.md](#) - One-shot core deploy script: [scripts/deploy_open5gs.sh](#) - UERANSIM configs used here: ~/UERANSIM/config/open5gs-gnb.yaml, ~/UERANSIM/config/open5gs-ue.yaml

Part A — Deploy the Open5GS 5G SA core (summary)

Full detail in open5gs-deployment.md; the essentials:

1. **Prereqs:** Docker, build tools, kernel headers; **Docker Compose v2**; and the **gtp5g kernel module** built + loaded on the host (Open5GS's 5G UPF data plane needs it — without it the UPF starts but won't forward user traffic):

```
git clone --depth 1 https://github.com/free5gc/gtp5g.git ~/gtp5g
cd ~/gtp5g && make && sudo make install && lsmod | grep gtp5g
```
2. **Deploy** with [herlesupreeth/docker_open5gs](#): clone, set .env (DOCKER_HOST_IP, MCC/MNC, and a non-clashing UE_IPV4_INTERNET), pull the GHCR image, then `docker compose -f sa-deploy.yaml up -d`. Or just run `./scripts/deploy_open5gs.sh`.
3. **Provision a subscriber** (see the auth note in Part C — provision it as **OP** type).

Core facts (this deployment):

Item	Value
PLMN / TAC	001 / 01, TAC 1
DNN / slice	internet / SST 1 (no SD)
Docker network	172.22.0.0/24 (gateway/host = 172.22.0.1)

Item	Value
AMF N2 (NGAP)	172.22.0.10:38412 (SCTP)
UPF N4 / N3	172.22.0.8:8805 / 172.22.0.8:2152
UE pool	10.45.0.0/16
WebUI	http://<host>:9999 (admin / 1423)
Subscriber	IMSI 001011234567895, K 8baf473f2f8fd09487cccbd7097c6862, OP 11111111111111111111111111111111, AMF 8000

Verify the core is healthy before attaching a UE:

```
sudo docker logs smf 2>&1 | grep "PFCP associated"      # SMF<->UPF N4 up
sudo docker logs upf 2>&1 | grep -E "pfcf_server|gtp_server"
sudo docker logs amf 2>&1 | grep "ngap_server"          # AMF NGAP listening
```

Part B — Attach UERANSIM (gNB + UE)

We use a **native UERANSIM build** on the host (~/.UERANSIM, built with make) — the same tool used for the other cores — rather than the bundled containers, so the setup is uniform.

Networking idea

Open5GS runs all NFs as containers on the Docker bridge 172.22.0.0/24. The host has IP 172.22.0.1 on that bridge and can reach every NF. So the gNB (on the host) binds 172.22.0.1 and talks to the AMF (172.22.0.10) and UPF (172.22.0.8) directly — no extra routing needed. The UE runs in its own network namespace on the host (see Problem 2).

UE (netns) — radio sim — gNB (host, 172.22.0.1) —N2—> AMF 172.22.0.10
└─N3 GTP-
U—> UPF 172.22.0.8 —> ogstun —NAT—> internet

gNB config — ~/.UERANSIM/config/open5gs-gnb.yaml

```
mcc: '001'
mnc: '01'
nci: '0x000000010'
idLength: 32
tac: 1
linkIp: 127.0.0.1      # radio-link sim to the UE (both on this host)
ngapIp: 172.22.0.1    # N2 to AMF (host's IP on the Open5GS docker bridge)
gtpIp: 172.22.0.1     # N3 GTP-U source (-> UPF 172.22.0.8)
amfConfigs:
  - address: 172.22.0.10 # Open5GS AMF NGAP
    port: 38412
slices:
  - sst: 1              # matches the subscriber/AMF default slice (SST 1, no SD)
ignoreStreamIds: true
```

UE config — ~/.UERANSIM/config/open5gs-ue.yaml (key fields)

```
supi: 'imsi-001011234567895'
mcc: '001'
```

[illegible]

Start it

```
cd ~/UERANSIM
sudo ./build/nr-gnb -c config/open5gs-gnb.yaml # expect: "NG Setup procedure is success
sudo ./build/nr-ue -c config/open5gs-ue.yaml # expect: "PDU Session establishment is
Success looks like a TUN interface coming up in a namespace, e.g. TUN interface[uesimtun0,
10.45.0.2] is up in namespace[uesimtun-001011234567895-internet-ps11].
```

Part C – Problems solved (and why)

Problem 1 — Authentication failure (MAC failure): OP vs OPc mismatch

Symptom: UE logs Authentication Reject received; AMF logs Authentication failure(MAC failure).

Cause: a 5G subscriber's operator key can be stored as either **OP** or **OPc** (OPc is derived from OP and K). The two ends must agree on both the value and the type. The open5gs-dbctl add IMSI K VALUE helper stores the value as **OPc**, but docker_open5gs's .env provides the value as **OP** (UE1_OP, and the bundled UE uses opType: OP). With the subscriber stored as OPc and the UE using OP, the computed keys differ → MAC check fails.

Fix: make both sides **OP**. Patch the subscriber so op is set and opc is null, and set the UE's opType: 'OP':

```
sudo docker exec mongo mongosh open5gs --quiet --eval '
  db.subscribers.updateOne({imsi:"001011234567895"},
    {$set:{"security.op":"11111111111111111111111111111111","security.opc":null}})'
```

(Equivalently, provision as OP via the WebUI, which lets you pick the type.) After this, registration succeeds. **Takeaway for fresh deploys:** decide OP vs OPc once and keep the subscriber and the UE consistent.

Problem 2 — UE traffic short-circuits when UE + gNB are on one host

Symptom (general): with the UE's TUN in the host's main network namespace, the UE's IP looks "local" to the host, so replies can be delivered straight to the TUN instead of going back through the radio→UPF path — breaking a clean end-to-end test.

Fix: run the UE in its own **network namespace** with `useNamespace: true`. UERANSIM then puts `uesimtun0` in a netns (e.g. `uesimtun-001011234567895-internet-ps11`). Run all UE traffic inside it:

```
NS=uesimtun-001011234567895-internet-psil
sudo ip netns exec $NS ping -c4 8.8.8.8
```

Problem 3 — DNS inside the UE namespace (for the speed test)

Symptom: name resolution fails inside the netns (no resolv.conf). **Fix:** give the namespace a resolver:

```
sudo mkdir -p /etc/netns/$NS
echo "nameserver 8.8.8.8" | sudo tee /etc/netns/$NS/resolv.conf
```

Problem 4 (non-issue, worth knowing) — SQN out of range

On first attach the UE may log Sending Authentication Failure due to SQN out of range, then immediately re-authenticate and succeed. This is the **normal** sequence-number resync between UE and core — no action needed.

Why Open5GS was easy where SD-Core was hard

Unlike the SD-Core BESS-UPF (DPDK + macvlan, which needed a route + MAC + ARP datapath fix for an external gNB and host-side NAT), the Open5GS UPF runs on a plain Docker bridge with gtp5g + ogstun and **NATs the UE pool to the internet itself**. So with Open5GS there is **no host NAT to add and no datapath patching** — once auth works, the data plane just flows.

Part D — Run the end-to-end tests

```
NS=uesimtun-001011234567895-internet-psil
# connectivity
sudo ip netns exec $NS ping -c4 -I uesimtun0 8.8.8.8
# throughput (iperf3 server on the host, reachable via the UPF NAT)
iperf3 -s -D
sudo ip netns exec $NS iperf3 -c <host-ip> -t 10 # uplink
sudo ip netns exec $NS iperf3 -c <host-ip> -t 10 -R # downlink
# real internet speed test
sudo ip netns exec $NS speedtest-cli --secure --simple
```

Results from this deployment:

Test	Result
Ping (UE → internet)	0% loss, ~7 ms
iperf3 downlink	107 Mbps
iperf3 uplink	97 Mbps
Ookla speed test	44 Mbps ↓ / 40 Mbps ↑

The single-connection iperf3 downlink is clean (~107 Mbps), confirming the UPF path comfortably carries ~100 Mbps; the speed test is bounded by the public server distance and the site's internet link, not the UPF.

Part E — Cheat sheet / fresh-deploy checklist

1. Build + load gtp5g; install Docker Compose v2.
2. `./scripts/deploy_open5gs.sh` (clones repo, configures `.env`, pulls image, brings up core).
3. Provision the subscriber as **OP** type (Problem 1).
4. Build UERANSIM (`cd ~/UERANSIM && make`) if not already built.
5. Write `open5gs-gnb.yaml` / `open5gs-ue.yaml` (Part B) — gNB binds the host's bridge IP 172.22.0.1, UE uses `opType: OP` + `useNamespace: true`.
6. Start `nr-gnb` then `nr-ue`; confirm NG Setup and PDU Session establishment succeed.
7. Add netns DNS (Problem 3), then run `ping` / `iperf3` / `speedtest` inside the UE netns.

Gotchas

- After a host reboot/kernel upgrade, rebuild gtp5g against the new headers (it auto-loads via `/etc/modules-load.d/gtp5g.conf`, but must exist for the running kernel).
- Keep subscriber **OP/OPc type** consistent with the UE config.
- The gNB's `ngapIp/gtpIp` must be the host's IP on the Open5GS bridge (172.22.0.1), not 127.0.0.1, so the containerized AMF/UPF can reach it.